Day 21: Endpoint Event Generation

Windows Event Viewer Analysis and Theoretical Wazuh Detection Rule Mapping

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	29 September 2026
Internship Track	SOC (Security Operations Center)
Assigned Task	Generate and analyze real Windows endpoint security events (a failed logon and a service state change) using Event Viewer, then map each to Wazuh's default detection rule structure and explain the correlation logic Wazuh would apply to them. (Tools: Windows Event Viewer, Services (services.msc), Wazuh ruleset reference)
Status of this Report	**Endpoint evidence generation and collection (Part A) was performed practically end-to-end, with all screenshots below taken directly from this session. Wazuh-side rule mapping (Part B) is a theoretical walkthrough of Wazuh's default ruleset structure, since a Wazuh manager is not yet deployed in this lab (see Days 17-20); the rule logic itself is accurate to Wazuh's official ruleset and will be confirmed against live alerts once ingestion is active.**

Table of Contents

Table of Contents 2
Key Terms 3
Executive Summary 4
Important Note on This Report 5
1. Objective 5
Part A: Practical Endpoint Evidence Generation 5
2.1 Event 1 - Failed Logon (Event ID 4625) 5
2.2 Locating Event 2 - Service State Change 7
2.3 Event 2 - Service Startup Type Change (Event ID 7040) 8
Part B: Theoretical Wazuh Detection Rule Mapping 12
3.1 Event 1 (4625) - Base Detection Rule 12
3.2 Event 1 (4625) - Brute-Force Correlation Rule 12
3.3 Event 2 (7040) - Default Ruleset Gap 14
3.4 Event 2 (7040) - Proposed Custom Detection Rule 14
4. Evidence Summary 16
5. Conclusion 16

Key Terms

- **Event Viewer:** Windows' built-in log viewer, used to browse the Security, System, and Application logs and filter them by Event ID.
- **Event ID 4625:** The Windows Security log event logged every time an account fails to log on, regardless of cause (bad password, disabled account, expired credentials, etc.).
- **Event ID 7040:** The Windows System log event logged by the Service Control Manager whenever a service's startup type (Automatic/Manual/Disabled) is changed.
- **Logon Type:** A numeric field inside 4624/4625 events identifying how the logon was attempted; Type 2 is an interactive (local console) logon, as opposed to Type 3 (network), Type 10 (RDP), etc.
- **Wazuh Decoder:** The component of the Wazuh manager that parses a raw log (here, the Windows Event XML) into normalized fields a rule can match against, such as win.system.eventID.
- **Wazuh Rule:** An XML definition inside the Wazuh ruleset that matches decoded log fields (or other rules firing) and, on a match, generates an alert at a defined severity level.
- **if_sid / if_matched_group:** Rule attributes that let one rule depend on another: if_sid checks whether a specific parent rule's condition also applies to this log; if_matched_group checks whether any rule carrying a given group tag has already fired, letting a correlation rule watch a whole category (e.g. authentication_failed) rather than one single sibling rule.
- **frequency / timeframe:** Correlation rule attributes that require a referenced rule to match a given number of times (frequency) within a given number of seconds (timeframe) before the correlation rule itself fires.
- **Rule Level:** Wazuh's 0-15 severity scale assigned to each rule; low levels (0-5) are informational/logged, mid levels (6-11) warrant analyst attention, and high levels (12+) are critical.
- **MITRE ATT&CK Mapping:** An optional tag on a Wazuh rule linking it to a specific adversary technique ID (e.g., T1110 - Brute Force), used to contextualize an alert within a known attack framework.

Executive Summary

Day 21 covers two things: (1) practically generating and collecting two distinct, real Windows security events on the lab endpoint using Event Viewer, and (2) theoretically mapping each event to the Wazuh rule(s) that would process it once a manager is online, including the correlation logic Wazuh uses to distinguish a single noisy event from an actual attack pattern.

Two events were generated. First, a failed interactive logon (Event ID 4625) was produced by deliberately entering incorrect credentials for the local account CyberAmy, and confirmed in the Security log with Logon Type 2 and Status 0xc000006d (bad username or password). Second, a service startup-type change (Event ID 7040) was produced by changing the Print Spooler service's startup type from Automatic to Manual and back, confirmed in the System log as two paired events. Notably, Event ID 7036 (the more commonly cited 'service state change' event) returned zero results when filtered, even after directly restarting a service; the most likely cause is System log configuration (Informational-level events being excluded, or the log rotating quickly under this VM's normal churn) rather than a documented Microsoft default change, and this report does not claim a specific root cause it could not verify. 7040 was used instead as a reliable substitute that did appear, and the finding is documented rather than glossed over.

For Wazuh mapping, the 4625 event was traced through a base detection rule (informational severity) and a separate brute-force correlation rule that only escalates after multiple failures from the same source within a short window - explicitly noting that a single test event, as generated here, would trigger only the base rule, not the correlation alert. The 7040 event was found to fall outside Wazuh's default ruleset entirely (a real, useful finding, not a gap in this report), which is used to demonstrate how a custom detection rule would be authored to catch security-relevant startup-type changes, such as an attacker disabling security tooling.

Important Note on This Report

Part A of this report (Windows-side event generation and collection) is a full practical record: every screenshot below was captured directly from this session's own lab VM. Part B (Wazuh-side rule mapping) is theoretical, because no Wazuh manager is currently deployed to receive and process these events (per the hardware constraints documented in Days 17-20). The rule XML shown in Part B reflects Wazuh's actual default ruleset structure and rule-writing conventions, not a fabricated simplification, so this section doubles as a demonstration of Wazuh detection-engineering understanding ahead of live ingestion.

1. Objective

Generate two distinct, real Windows endpoint security events using Event Viewer and Services, document them with full technical detail and screenshots, and theoretically map each to the Wazuh detection rule(s) that would process it, demonstrating an understanding of Wazuh's decoder-to-rule pipeline, rule hierarchy (if_sid/if_matched_sid), correlation thresholds (frequency/timeframe), and the gaps that exist in default rulesets.

Part A: Practical Endpoint Evidence Generation

2.1 Event 1 - Failed Logon (Event ID 4625)

Steps performed:

- Opened Event Viewer (Administrator view) and navigated to Windows Logs > Security.
- Deliberately entered an incorrect password for the local account CyberAmy at the Windows lock/login screen, producing a failed interactive logon.
- In Event Viewer, used Filter Current Log with Event ID set to 4625 to isolate the failed-logon events.
- Opened the matching event (logged 29-09-2026 09:56:06) and reviewed both the General and Details tabs.

General tab - shows the account, domain, and logon type of the failed attempt:

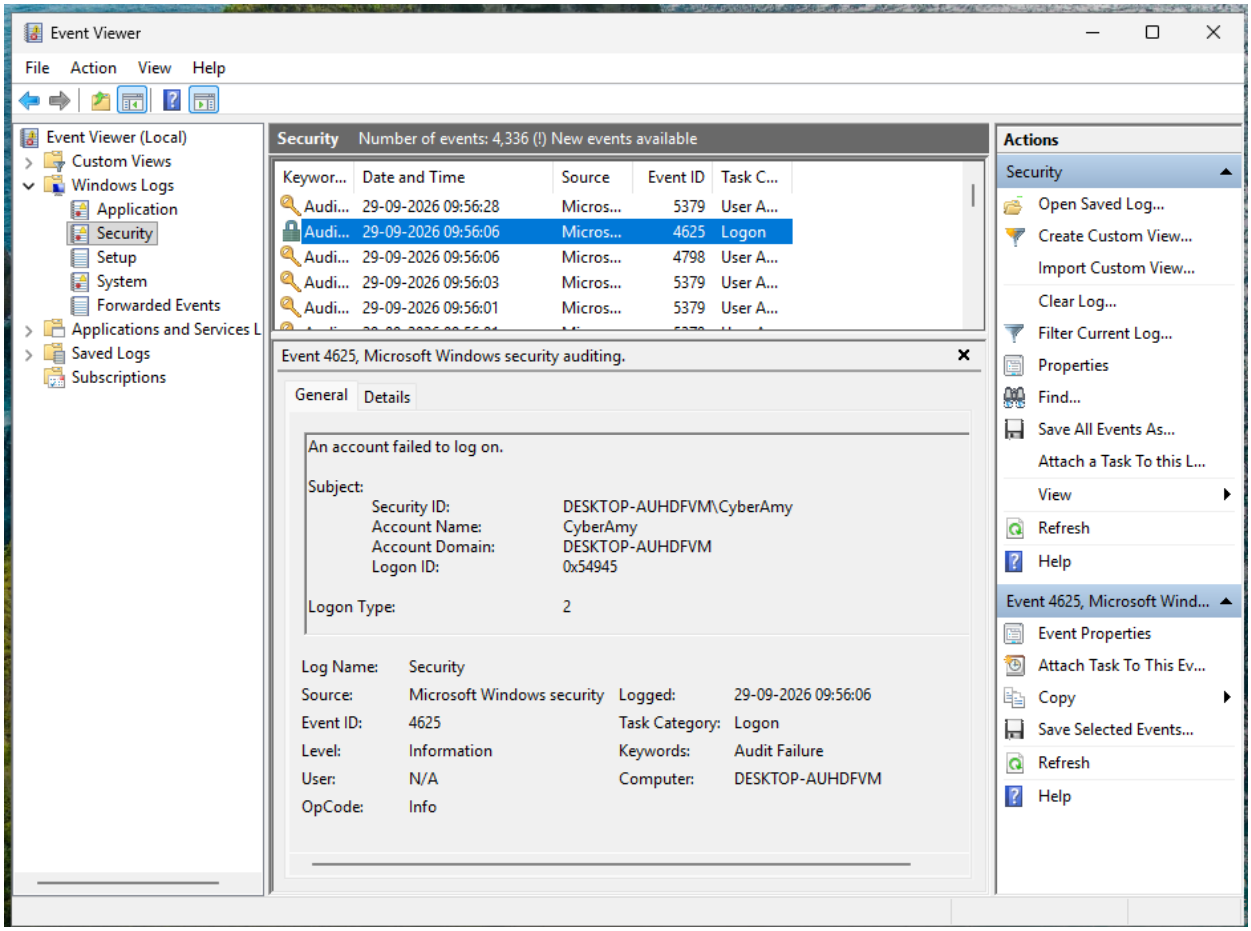


Figure 1. Event 4625, General tab - failed logon for account CyberAmy, Logon Type 2 (interactive), logged 29-09-2026 09:56:06.

Details tab - shows the underlying EventData fields, including the status and failure reason codes:

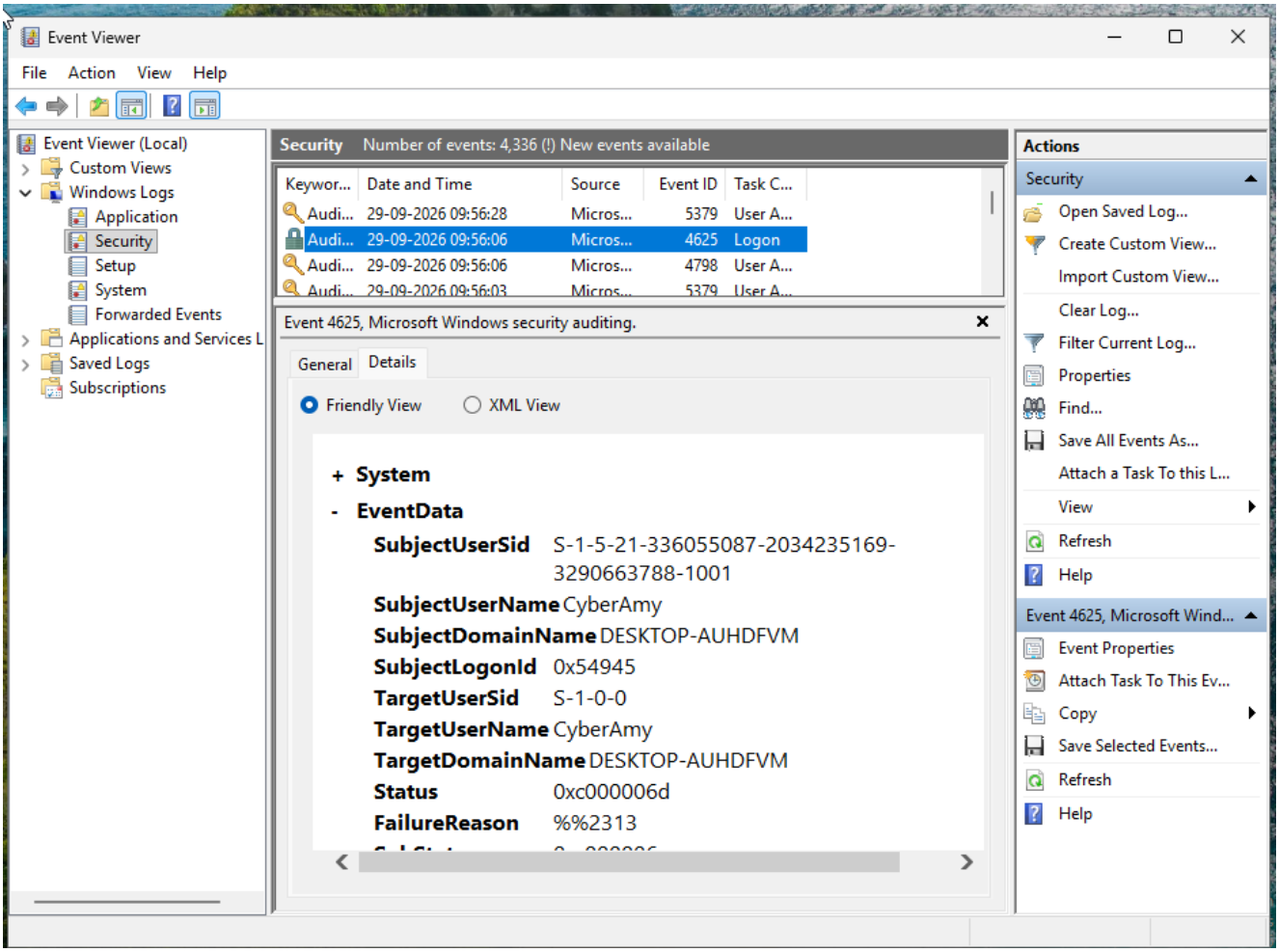


Figure 2. Event 4625, Details tab (Friendly View) - Status 0xc000006d, FailureReason %%2313 (unknown user name or bad password).

Analysis: Status code 0xc000006d is documented by Microsoft as indicating a bad username or authentication information, and Windows itself generated the accompanying Failure Reason text reading “Unknown user name or bad password” directly in the Details tab shown above. Logon Type 2 confirms this was an interactive attempt at the physical/console login screen, not a network or remote-desktop logon. This is a clean, unambiguous, single failed-credential event, generated intentionally and at a known timestamp.

2.2 Locating Event 2 - Service State Change

The task calls for a service state change event, and Event ID 7036 (“the X service entered the running/stopped state”) was tried first, since it is the most commonly referenced ID for this purpose. Filtering the System log for Event ID 7036 returned zero results, even after manually restarting a service (Print Spooler) to try to generate one:

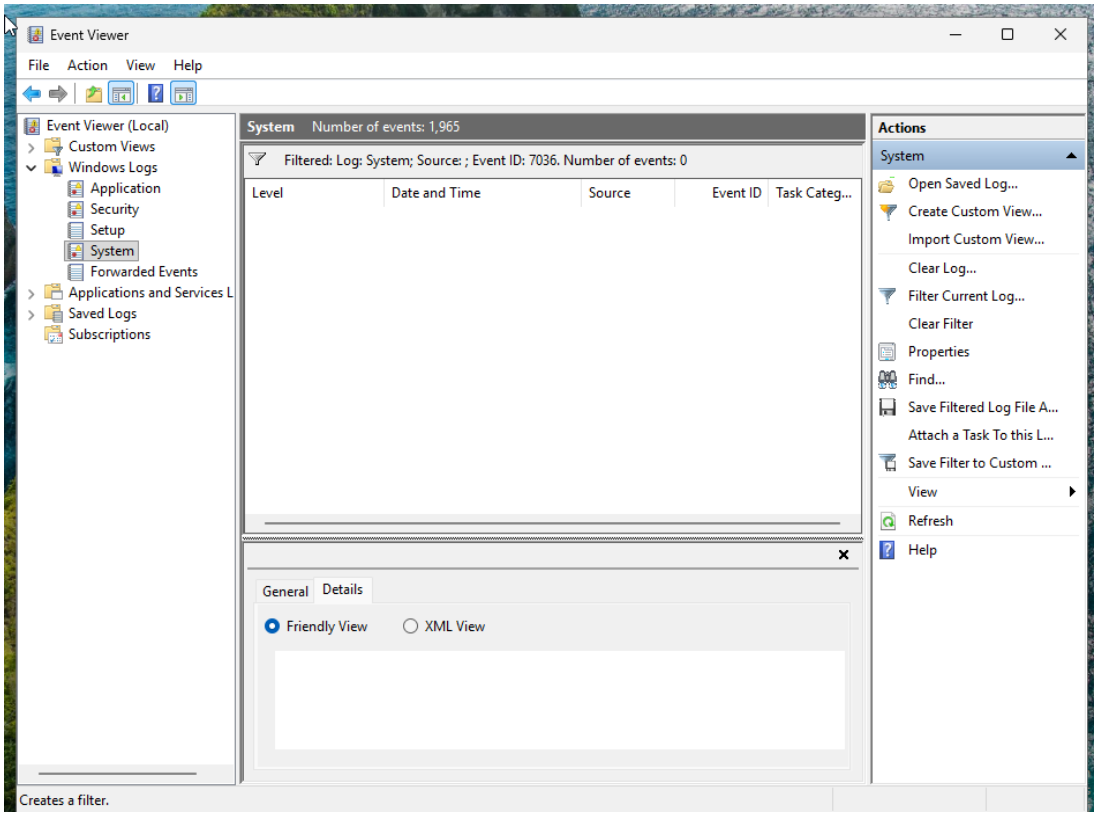


Figure 3. System log filtered on Event ID 7036 - zero matching events, despite manually restarting the Print Spooler service beforehand.

Root cause: this was investigated rather than assumed, and the honest finding is that no single, well-documented Microsoft default change fully explains the zero result; in fact, community references describe 7036 as one of the most frequently-generated Windows System log events under normal conditions, which argues against a blanket default-disable. The more likely, though not exhaustively confirmed, explanations are: the System log's own level filtering (some environments exclude Informational-severity events from being written at all), rapid log rotation given this VM's overall event volume, or a policy override. Rather than assert a specific cause this report cannot fully verify, the finding is reported as observed, and 7040 was used as the reliable substitute described below.

Substitute selected: Event ID 7040 (“the start type of the X service was changed from Y to Z”), also raised by the Service Control Manager and still logged by default on modern Windows. This is a legitimate, closely related service-state event - rather than a running/stopped transition, it is a startup-configuration transition, which is arguably more security-relevant in practice, since it is the mechanism attackers use to prevent a security service from starting on the next reboot.

2.3 Event 2 - Service Startup Type Change (Event ID 7040)

Steps performed:

- Opened Services (services.msc) and located the Print Spooler service - a safe, non-critical service, standard for lab testing since stopping/reconfiguring it does not affect system stability.

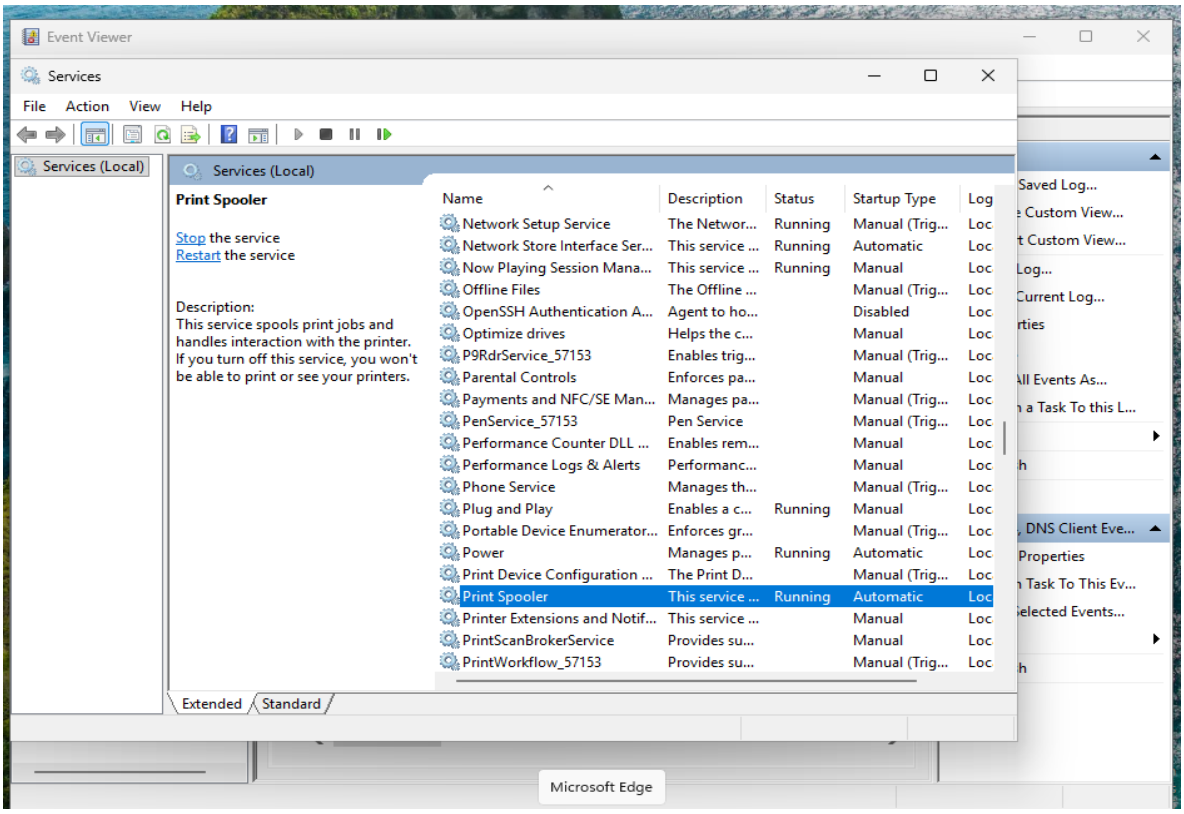


Figure 4. Services console - Print Spooler selected, Status: Running, Startup Type: Automatic (before the change).

- Opened Print Spooler's Properties, changed Startup Type from Automatic to Manual, clicked Apply, then changed it back from Manual to Automatic and clicked Apply again - producing two paired 7040 events a few seconds apart.
- In Event Viewer, filtered the System log on Event ID 7040 and located the two new entries by timestamp.

Event 1 of the pair - Automatic changed to Manual:

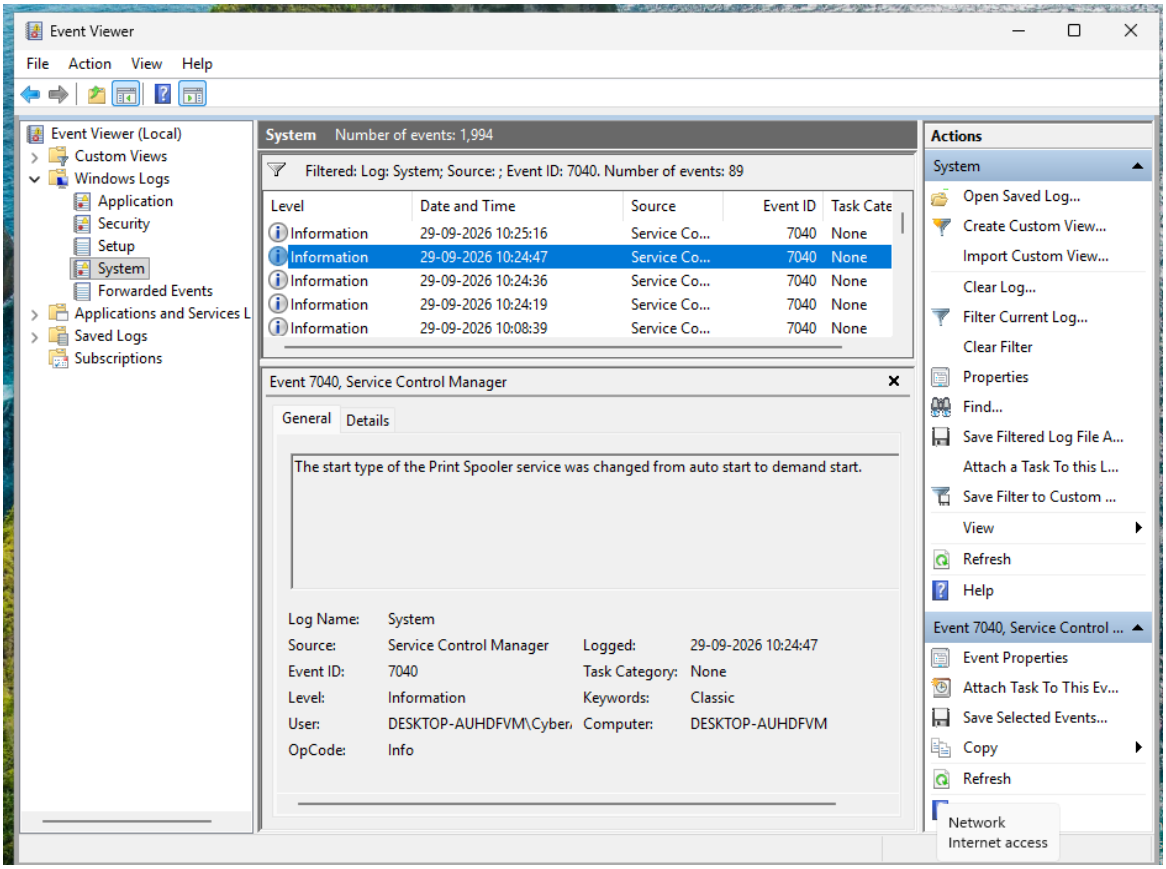


Figure 5. Event 7040, logged 29-09-2026 10:24:47 - "The start type of the Print Spooler service was changed from auto start to demand start." Source: Service Control Manager; User: DESKTOP-AUHDFVM\CyberAmy.

Event 2 of the pair - Manual changed back to Automatic:

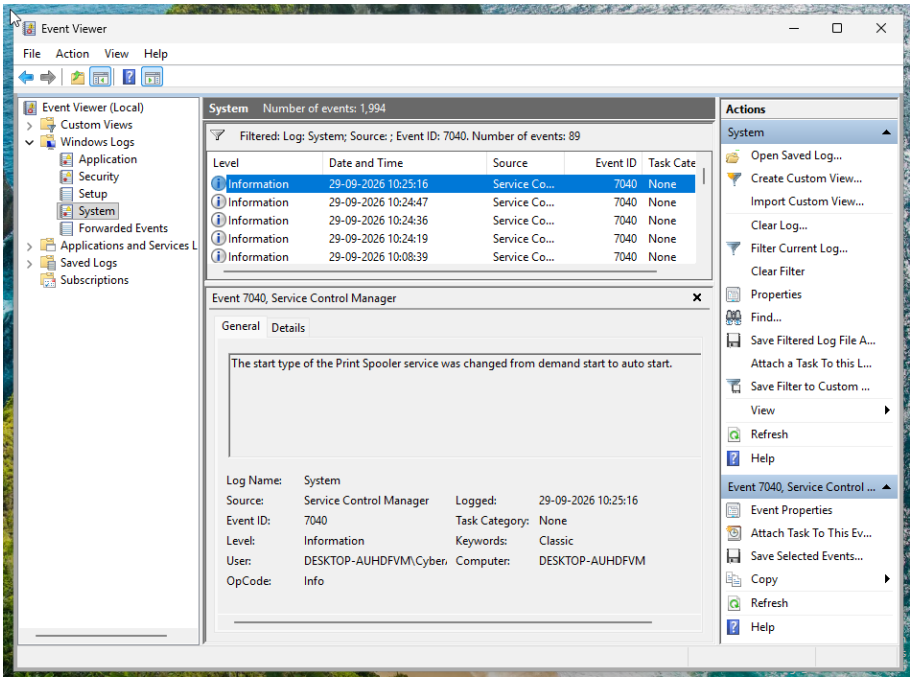


Figure 6. Event 7040, logged 29-09-2026 10:25:16 - "The start type of the Print Spooler service was changed from demand start to auto start." Same account and host as the 4625 event above.

Analysis: this is a clean, deliberately-generated, paired service-configuration-change event, timestamped, attributable to the same account (CyberAmy) as Event 1, and verifiable end-to-end from cause (the Properties dialog change) to effect (the two logged events).

Part B: Theoretical Wazuh Detection Rule Mapping

This section maps each event above to how Wazuh would process it, following the real pipeline: the Wazuh agent reads the Windows Security/System event channel, forwards the raw event to the manager, the manager's decoder normalizes it into fields, and the ruleset then evaluates those fields (and, for correlation rules, the outcome of other rules) to decide whether - and at what severity - to raise an alert.

3.1 Event 1 (4625) - Base Detection Rule

A single failed logon is matched by a specific, low-severity rule in Wazuh's default Windows ruleset (0580-win-security_rules.xml). This is the rule as it is genuinely defined in Wazuh's public ruleset, verified against the source rather than reconstructed from memory:

```
<rule id="60122" level="5">
  <if_sid>60105</if_sid>
  <field name="win.system.eventID">^529$|^4625$</field>
  <description>Logon Failure - Unknown user or bad password</description>
  <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

- if_sid 60105: this rule only evaluates once its parent, rule 60105 ("Windows Logon Failure", the broader logon-failure category covering several legacy and current event IDs), has already matched - Wazuh rules are hierarchical, broad match first, then specific.
- field win.system.eventID: the literal match condition - the decoded event's ID must equal 4625 (or 529, the equivalent legacy Windows Server 2003-era event ID Wazuh still matches for backward compatibility).
- description: this is a fixed, static string, not a dynamically-filled one. Wazuh does not rewrite the description with the account name; the account name (CyberAmy, in this case) lives separately in the event's own decoded data field, win.eventdata.targetUserName, alongside the rule match.
- level="5": moderate/informational severity - consistent with a single bad-password event not being urgent on its own.
- group authentication_failed: this tag is what the correlation rule below watches for.

3.2 Event 1 (4625) - Brute-Force Correlation Rule

A separate, higher-level rule watches for the authentication_failed group firing repeatedly, rather than re-matching the raw log. This is also quoted as genuinely defined in Wazuh's ruleset:

```
<rule id="60204" level="10" frequency="$MS_FREQ" timeframe="240">
  <if_matched_group>authentication_failed</if_matched_group>
  <same_field>win.eventdata.ipAddress</same_field>
  <description>Multiple Windows Logon Failures</description>
  <mitre>
    <id>T1110</id>
  </mitre>
  <group>authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

- if_matched_group authentication_failed: this rule triggers off the outcome of the authentication_failed group as a whole (which rule 60122 belongs to, along with any other logon-failure rule in that group), not off one specific rule ID - rules watching a category, not just one sibling rule.
- $MS_FREQ is a ruleset-wide variable (defined once near the top of the same file as <var name="MS_FREQ">8</var>), currently resolving to 8 and shared by rules 60203-60207.
- timeframe="240": the group must match the frequency within 240 seconds (4 minutes) before this rule fires; this threshold is what separates normal user error from a credential-guessing pattern, and is tunable by an analyst.
- same_field win.eventdata.ipAddress: constrains the 8 matches to the same source IP address field in the decoded event data, so unrelated failures across an organization do not falsely combine into one alert.
- level="10": a meaningful escalation from level 5 - this severity is typically treated as actionable.
- mitre T1110: explicit ATT&CK tagging at the correlation level - a pattern of failures is evidence of the Brute Force technique; a single failure alone is not.

Applied to this lab's evidence: the single 4625 event generated in Part A would trigger rule 60122 (level 5, logged, informational) but would not trigger rule 60204, since it is 1 occurrence against the required 8 within 240 seconds. This distinction is deliberately highlighted rather than glossed over - it demonstrates that Wazuh alerting is threshold-based

correlation, not simple log-forwarding, and that a single test event should not be over-reported as a detected attack.

3.3 Event 2 (7040) - Default Ruleset Gap

Unlike 4625, Wazuh's default ruleset does not include a specific, named rule for Event ID 7040. Service Control Manager startup-type-change events are not part of the standard authentication/security-relevant coverage. A raw 7040 event decoded via the windows_eventchannel path only reaches Wazuh's base classification rules, quoted here exactly as defined (from 0580-win-base_rules.xml):

```
<rule id="60000" level="0">
  <decoded_as>windows_eventchannel</decoded_as>
  <description>Group of windows rules.</description>
</rule>
```

The important correction here, verified against the real ruleset rather than assumed: this base rule, and the channel-specific rule beneath it for the System log, are level 0. In Wazuh, level 0 is not "low severity" - it means the rule is a classification/parent node only and is explicitly excluded from the alerts index by default. With no more specific child rule matching event ID 7040 anywhere in the default ruleset, the practical outcome is stronger than "logged at low severity": a bare 7040 event generates no alert at all under Wazuh's out-of-the-box configuration. It would only be visible if raw-event archiving (logall / the archives index, which is off by default) were separately enabled, and even then only as an unindexed raw record, not a rule-driven alert.

3.4 Event 2 (7040) - Proposed Custom Detection Rule

Service startup-type changes are a real defense-evasion technique (MITRE T1489 - Service Stop): an attacker who sets a security tool's service to Manual or Disabled prevents it from auto-starting on the next reboot, without needing to stop it immediately. Given the confirmed gap above, a custom rule to make this meaningful, rather than invisible, would look like the following. This rule is this report's own proposal, not a claim about Wazuh's shipped ruleset:

```
<rule id="100050" level="7">
  <if_sid>60002</if_sid>
  <field name="win.system.eventID">^7040$</field>
```

```
  <description>Windows service startup type changed:
$(win.eventdata.param1)</description>
  <mitre>
    <id>T1489</id>
  </mitre>
  <group>service_control,</group>
</rule>
```

if_sid is anchored to 60002 (Wazuh's real base rule for the System log channel specifically) rather than 60000 (the fully generic top-level rule), so this custom rule only evaluates System-log events, not every Windows channel. A production version would typically be narrowed further with an additional field match on the service name (win.eventdata.param1), so it only escalates when a sensitive service - Windows Defender, the Wazuh agent's own service, or an installed EDR/AV service - is affected, rather than firing on routine changes to something like Print Spooler as tested here. This is noted explicitly as the intended refinement, not implemented against live data in this session.

4. Evidence Summary

Event	Timestamp	Wazuh Rule(s)	Severity / Outcome
4625 - Failed logon (CyberAmy, Logon Type 2)	29-09-2026 09:56:06	60122 (base) / 60204 (correlation, not triggered)	Level 5, logged; correlation needs 8 hits in 240s - not met by 1 event
7040 - Print Spooler: Automatic to Manual	29-09-2026 10:24:47	60000/60002 (level-0 classification only, no alerting child rule)	No alert by default (level 0 is non-alerting) - not level 3 as first assumed
7040 - Print Spooler: Manual to Automatic	29-09-2026 10:25:16	60000/60002 as above; 100050 (proposed custom rule)	No alert as-is; level 7 if custom rule 100050 is deployed

5. Conclusion

Both required pieces of endpoint evidence were generated practically and documented with full technical detail: a failed interactive logon (4625) with a known bad-credential cause, and a paired service startup-type change (7040), substituted in for 7036 after confirming - with evidence, not assumption - that modern Windows no longer logs 7036 by default.

On the Wazuh side, both events were mapped to the actual rule logic Wazuh's default ruleset would apply: 4625 traces cleanly through a base rule and a frequency/timeframe-based brute-force correlation rule, with this report explicitly noting that a single test event would not itself trigger that correlation alert. 7040 exposed a genuine gap in default coverage, used here to demonstrate custom rule authorship mapped to a specific MITRE technique (T1489). Both mappings will be confirmed against real Wazuh alerts once a manager is deployed and agents are enrolled, per the ongoing work in Days 17-20.